

Build CarModel and CarMake Django Models



Estimated time: 90 minutes

A dealership typically manages cars from one or more makers or manufacturers, and customers should be allowed to review the cars they purchased from a dealer.

In this lab, you will create the **CarModel** and **CarMake** models in the Django app.

- A **car model** includes basic information, such as its make, year, type, and dealer ID.
- A **car make** includes basic information, such as name and description.

Environment Setup

If your lab workspace has not been reset, you can skip this section. Else continue from what you have done previously with the following:

- `git clone` or pull from your created GitHub repository
- Run the command given below to set up the Django environment

 bash 

```
cd /home/project/xrwvm-fullstack_developer_capstone/server
pip install virtualenv
virtualenv djangoenv
source djangoenv/bin/activate
```

 Run

- Install the required packages by running the command given below

 bash 

```
python3 -m pip install -U -r requirements.txt
```

 Run

- Run the command given below to perform model migrations

 bash 

```
python3 manage.py makemigrations
python3 manage.py migrate
python3 manage.py runserver
```

 Run

Steps to Build CarModel and CarMake Models

In this section, you need to create two new models in `server/djangoapp/models.py`:

1. A `CarMake` model to save some data about a car's make.
2. A `CarModel` model to save some data about a car's model.
 - Create a car make Django model `class CarMake(models.Model)`:
 - Name
 - Description
 - Any other fields you would like to include in a car make
 - A `__str__` method to print a car make object

► [Click here for a sample](#)

- Create a car model Django model `class CarModel(models.Model)`
 - Many-to-one relationship to `CarMake` model (One car make can have many car models, using a ForeignKey field)
 - Dealer ID (IntegerField) refers to a dealer created in Cloudbant database
 - Name
 - Type (CharField with a choices argument to provide limited choices such as Sedan, SUV, and Wagon)
 - Year (IntegerField)
 - Any other fields you would like to include in a car model
 - A `__str__` method to print the car make and car model object

► [Click here for a sample](#)

- You need to register the `CarMake` and `CarModel` on the admin site so you can conveniently manage their content, such as perform CRUD operations. Refer to the Lab: [Django Admin Site](#) for more details.

► [Click here for sample code](#)

- Run migrations for the models



bash



```
python3 manage.py makemigrations
python3 manage.py migrate --run-syncdb
```

▶ Run

Note: The `--run-syncdb` allows creating tables for apps without migrations.

Refer to the Lab: Django ORM for more details:

[CRUD on Django Model Objects](#)

Steps to Register the CarMake and CarModel Models with the Admin Site

- First, you must have superuser access on the admin site (if not created before).
- Next, use `root` as the user name and `root` as the password for your reviewer to log in to your app.
- Open a new terminal and build your client by running the commands given below

 bash 

```
cd /home/project/xrwvm-fullstack_developer_capstone/server/  
npm install  
npm run build
```

 Run

- Start the server from the terminal where Django application was running if it is not already running.

 plaintext 

```
python3 manage.py runserver
```

- Select the button below to launch the admin page to login with the root credentials.

[Django Admin](#)

▶ [Click here if you get an error](#)

► If the page doesn't open, select here to open the URL manually.

Assessment:

For Option 1: AI-Graded Submission and Evaluation and **For Option 2:** Peer-Graded Submission and Evaluation

- After you log in to the admin site, take a screenshot and name it as `admin_login.jpeg` or `admin_login.png`.
- In addition, you may want to log out as the admin user. You will be redirected to the admin login page again. Take a screenshot and name it `admin_logout.jpeg` or `admin_logout.png`.
- Open `djangoapp/views.py`, import `CarMake` and `CarModel` in it after the other import statements in the beginning of the file, and add a method to get the list of cars by including the code below:



python



```
from .models import CarMake, CarModel
```



py



```
def get_cars(request):
    count = CarMake.objects.filter().count()
    print(count)
    if(count == 0):
        initiate()
    car_models = CarModel.objects.select_related('car_make')
    cars = []
    for car_model in car_models:
        cars.append({"CarModel": car_model.name, "CarMake":
    return JsonResponse({"CarModels":cars})
```

- Open `server/djangoapp/urls.py` and add the path for `get_cars` in it.



python



```
path(route='get_cars', view=views.get_cars, name='getc
```

- Open `server/djangoapp/populate.py` and paste the code given below to populate data in your database. The data is populated when the first call is made to `get_cars`, if the CarModel is empty. If you wish to add data manually, skip this step.



python



```
from .models import CarMake, CarModel

def initiate():
    car_make_data = [
        {"name": "NISSAN", "description": "Great cars. Japane
        {"name": "Mercedes", "description": "Great cars. Germ
        {"name": "Audi", "description": "Great cars. German t
        {"name": "Kia", "description": "Great cars. Korean te
        {"name": "Toyota", "description": "Great cars. Japane
    ]

    car_make_instances = []
    for data in car_make_data:
        car_make_instances.append(CarMake.objects.creat

# Create CarModel instances with the corresponding CarM
car_model_data = [
    {"name": "Pathfinder", "type": "SUV", "year": 2023, "ca
    {"name": "Qashqai", "type": "SUV", "year": 2023, "car_m
    {"name": "XTRAIL", "type": "SUV", "year": 2023, "car_ma
    {"name": "A-Class", "type": "SUV", "year": 2023, "car_m
    {"name": "C-Class", "type": "SUV", "year": 2023, "car_m
    {"name": "E-Class", "type": "SUV", "year": 2023, "car_m
    {"name": "A4", "type": "SUV", "year": 2023, "car_make":
    {"name": "A5", "type": "SUV", "year": 2023, "car_make":
    {"name": "A6", "type": "SUV", "year": 2023, "car_make":
    {"name": "Sorrento", "type": "SUV", "year": 2023, "car_
    {"name": "Carnival", "type": "SUV", "year": 2023, "car_
    {"name": "Cerato", "type": "Sedan", "year": 2023, "car_
    {"name": "Corolla", "type": "Sedan", "year": 2023, "car
    {"name": "Camry", "type": "Sedan", "year": 2023, "car_m
    {"name": "Kluger", "type": "SUV", "year": 2023, "car_ma
    # Add more CarModel instances as needed
]

for data in car_model_data:
    CarModel.objects.create(name=data['name'], car_
```

- Alternatively, if you wish to add the Car makes and models manually, you can go to the admin page and add them whenever it is required. Note that you will only be allowed to choose from one of the makes and models in these tables to post a review in the later part of the

► If the page doesn't open, select here to open the URL manually.

- Select the **Get Cars** button below to check the list of cars added.

If you chose to populate, it will show five CarMakes and fifteen Car Models, three under each make. Else it will show what you added.

Get Cars

► If the page doesn't open, select here to open the URL manually.

Assessment:

For Option 1: AI-Graded Submission and Evaluation: Run the below command in a new terminal. Copy and paste the terminal output along with the command and save it in a text file named **getallcarmakes** for the final project submission and evaluation.



plaintext



```
curl -X GET "<Django-URL>/djangoapp/get_cars"
```

Note: Replace the Django URL with your actual URL before running the command.

For Option 2: Peer-Graded Submission and Evaluation:

- Take a screenshot of the car and model and name it **cars.jpeg** or **cars.png**.
- Select the **Django Admin** button below and take a screenshot of the car models and save it as **car models.png** or **car models.jpeg**.

Commit your Updated Project to GitHub

Commit all updates to the GitHub repository to can save your work.

If you need to refresh your memory on how to commit and push your code to GitHub in the Theia lab environment, refer to the lab [Working with git in the lab environment](#)

Summary

Congratulations on completing the **Lab: Build CarModel and CarMake Django Models!** In this lab, you have created the `CarModel` and `CarMake` models in your Django app.

Author(s)

[Lavanya](#)

Other Contributor(s)

Upkar Lidder Yan Luo Priya

© Copyright IBM Corporation. All rights reserved.

Build CarModel and CarMake Django Models



Estimated time: 90 minutes

A dealership typically manages cars from one or more makers or manufacturers, and customers should be allowed to review the cars they

In this lab, you will create the `CarModel` and `CarMake` models in the Django app.

- A `car model` includes basic information, such as its make, year, type, and dealer ID.
- A `car make` includes basic information, such as name and description.

[Environment Setup →](#)